

Contents

ParaView I/O — Codebase Walkthrough	1
READ THIS FIRST: Opening Phase 6 Output on a macOS ParaView	1
macOS-specific gotcha: DYLD_LIBRARY_PATH causes a segfault	2
One-time setup (run this once, then never again)	2
Why this song and dance is necessary (3-step root cause)	3
Overview	3
How the split-bulk-solutions refactor works, and how to adjust outputs	3
Three layers a user can adjust	4
Quick recipes	6
Architecture	6
Three back ends, three switches	6
Key files	6
Data flow per write cycle (parallel build, HDF5 mode)	7
External dependencies	8
Call Graph (critical path, TPV* driver)	8
Call graph (BP5)	9
Detailed Walkthrough	9
seas::ParaViewOutput<MeshType> (paraview_output.hpp:69 — 2093 lines)	9
vtkhdf::FaultHDFState and WriteFaultPackHdf (fault_vtkhdf_writer.hpp)	12
vtu::LocalFaultPack / GatherFaultPackToRoot (fault_vtu_binary.hpp)	12
MFEM-level ParaViewHDFDataCollection (fem/datacollection.{hpp,cpp})	13
MFEM-level VTKHDF (mesh/vtkhdf.{hpp,cpp})	13
build_frontera.sh + Phase 6 sbatch glue	14
Phase chronology (from git log, document/io_dev/PLAN_*.md)	14
Conventions	15
Gotchas	16
Open Questions	17

ParaView I/O — Codebase Walkthrough

Date: 2026-05-11 **Scope:** ParaView visualization output for the seas miniapp (BP5 + TPV102/104/205 drivers), including the Phase 6 VTKHDF+ZFP compaction work. Covers the wrapper `seas::ParaViewOutput<MeshType>`, its three underlying collections, the MFEM-level VTKHDF substrate, the ZFP compression layer, and the driver-side glue. **Audience:** Computational scientist who has never read this code before. The math is elsewhere — this document is exclusively about I/O.

READ THIS FIRST: Opening Phase 6 Output on a macOS ParaView

The `.vtkhdf` files this codebase produces are ZFP-compressed. ParaView 5.11+ knows how to *render* them but does NOT bundle the H5Z-ZFP plugin needed to *decompress* them. Out of the box, opening one of our files in ParaView fails with:

```
HDF5: object 'Points' doesn't exist
HDF5: object 'Values' doesn't exist
Cannot read the Points_0_array from file
ERROR: vtkHDFReader: Cannot read the Points array
```

That's not a file corruption — it's the HDF5 read path silently failing because the ZFP filter isn't registered. The plugin (`libh5zzfp.dylib`, filter id 32013) must be **discoverable at runtime via `HDF5_PLUGIN_PATH`** AND its dependencies must resolve without conflicting with ParaView's bundled HDF5.

macOS-specific gotcha: DYLD_LIBRARY_PATH causes a segfault

The plugin from the mfem-dev conda env has @rpath/libzfp.1.dylib and @rpath/libhdf5.310.dylib as dependencies. Setting DYLD_LIBRARY_PATH=<conda-env>/lib to satisfy those drags conda's libhdf5.310.dylib into the ParaView process *alongside* ParaView's own bundled libhdf5.310.dylib — two copies of HDF5 in one process -> instant segfault on launch.

DYLD_FALLBACK_LIBRARY_PATH works only sometimes (macOS SIP / amfi may strip it). The reliable solution is to install_name_tool the plugin so its dependencies point at **absolute paths** baked in, then re-sign with an ad-hoc signature so Gatekeeper accepts it.

One-time setup (run this once, then never again)

```
# 1. Build a private, self-contained copy of the plugin with absolute  
#    dependency paths.
```

```
mkdir -p ~/.h5z-zfp-deps/plugin  
cp /Users/chunhuizhao/miniforge/envs/mfem-dev/plugin/libh5zzfp.dylib \  
  ~/.h5z-zfp-deps/plugin/  
install_name_tool \  
  -change @rpath/libzfp.1.dylib \  
    /Users/chunhuizhao/miniforge/envs/mfem-dev/lib/libzfp.1.dylib \  
  -change @rpath/libhdf5.310.dylib \  
    /Applications/ParaView-6.0.0-RC3.app/Contents/Libraries/libhdf5.310.dylib \  
  ~/.h5z-zfp-deps/plugin/libh5zzfp.dylib
```

```
# 2. Re-sign the modified dylib — install_name_tool invalidates the  
#    code signature and macOS kills any process trying to load it.
```

```
codesign --force --sign - ~/.h5z-zfp-deps/plugin/libh5zzfp.dylib
```

```
# 3. Make HDF5_PLUGIN_PATH and a 'paraview' alias permanent in zsh.
```

```
echo 'export HDF5_PLUGIN_PATH=$HOME/.h5z-zfp-deps/plugin' >> ~/.zshrc  
echo 'alias paraview="/Applications/ParaView-6.0.0-RC3.app/Contents/MacOS/paraview"' >> ~/.zshrc  
source ~/.zshrc
```

After that, opening any of our .vtkhdf files is a one-liner:

```
paraview fault_surface.vtkhdf &
```

Note: ~/.zshrc covers only the terminal. To also open .vtkhdf by double-clicking in Finder (or via Spotlight), install a LaunchAgent that sets HDF5_PLUGIN_PATH system-wide:

```
mkdir -p ~/Library/LaunchAgents  
cat > ~/Library/LaunchAgents/com.user.hdf5pluginpath.plist <<'EOF'  
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"  
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">  
<plist version="1.0">  
<dict>  
  <key>Label</key><string>com.user.hdf5pluginpath</string>  
  <key>ProgramArguments</key>  
  <array>  
    <string>launchctl</string><string>setenv</string>  
    <string>HDF5_PLUGIN_PATH</string>  
    <string>/Users/chunhuizhao/.h5z-zfp-deps/plugin</string>  
  </array>  
  <key>RunAtLoad</key><true/>  
</dict>  
</EOF>
```

```

</plist>
EOF
launchctl load ~/Library/LaunchAgents/com.user.hdf5pluginpath.plist

```

Why this song and dance is necessary (3-step root cause)

1. **ZFP compression is non-optional.** Without it, BP5 production output is hundreds of GB; the whole point of Phase 6 was to bring it under a GB.
2. **HDF5 filters are dlopen'd at decompression time** from HDF5_PLUGIN_PATH. The plugin is not linked into ParaView.
3. **macOS Gatekeeper + the two-HDF5-in-one-process trap** make the obvious environment-variable solutions fail (segfault) or get blocked (kill). The `install_name_tool + codesign --sign - recipe` is the only reliable workaround for an unmodified ParaView .app.

If you ever reinstall ParaView or update conda's `libhdf5`, the absolute paths inside `~/h5z-zfp-deps/plugin/libh5zzfp.dylib` will become stale — re-run the one-time setup. The plugin's `otool -L` output should show no `@rpath/...` entries when correctly set up.

Overview

A single SEAS run on Frontera produces three ParaView artefacts (renamed per document/`io_dev/PLAN_split_bulk_sol05-12.md` — older runs and out-of-tree sbatch may still show the legacy names in parentheses):

File	Contents	Purpose
<code><output>/ParaView/kinematics</code> (was <code>volume.vtkhdf</code>)	Kinematics — velocity (TPV*) or displacement (BP5) + <code>mpi_rank</code>	Animation in ParaView 5.11+
<code><output>/ParaView_bulk/stress</code> (was <code>wave_bulk.vtkhdf</code>)	Stress — full symmetric tensor <code>sigma_xx, _yy, _zz, _xy, _xz, _yz</code>	Wavefield diagnostics; TPV* only (BP5 has no stress file — quasi-dynamic)
<code><output>/fault.vtkhdf</code> (was <code>fault_surface.vtkhdf</code>)	Fault interface — slip, slip-rate, traction, state on the 2D fault	The science output

Each collection has its own writer (back-end), cadence, compression tolerance, and ParaView-side time series. The wrapper `seas::ParaViewOutput<MeshType>` owns the kinematics + fault collections; the stress collection is a second instance of `seas::ParaViewOutput` created next to it in the TPV drivers.

The historical context (per document/`io_dev/PLAN_paraview_compaction_2026-04-28.md`): a 250-yr BP5 production run on Frontera at `np=800` used to emit **3.9 million** per-rank-per-cycle ASCII VTU files into one Lustre directory, breaking `du`, `ls`, `tar`, and `rsync`. The compaction work replaced that with single-file VTKHDF (HDF5 + chunked + ZFP-compressed), reducing both file count (millions -> 3) and bytes (~25 GB -> ~200-400 MB) per run.

How the split-bulk-solutions refactor works, and how to adjust outputs

Before the 2026-05-12 refactor, the TPV drivers wrote two volume files (`volume.vtkhdf` + `wave_bulk.vtkhdf`) that BOTH carried velocity + `mpi_rank` — a byte-for-byte duplication whenever the cadences matched (the default). The primary file ALSO carried 12 fault projections that duplicated data already in `fault_surface.vtkhdf`. Three problems:

1. ~50 % of `wave_bulk.vtkhdf` was a duplicate of `volume.vtkhdf`.

2. The 12 fault projections in `volume.vtkhdf` duplicated `fault_surface.vtkhdf`.
3. The names (“volume”, “wave_bulk”, “fault_surface”) were physically vague.

The refactor (`document/io_dev/PLAN_split_bulk_solutions_2026-05-12.md`) does three things:

Aspect	Old	New
Names	<code>volume.vtkhdf /</code> <code>wave_bulk.vtkhdf /</code> <code>fault_surface.vtkhdf</code>	<code>kinematics.vtkhdf /</code> <code>stress.vtkhdf /</code> <code>fault.vtkhdf</code>
Stress fields	3 components (<code>sigma_yy</code> , <code>sigma_xy</code> , <code>sigma_xz</code>)	6 — full symmetric tensor (<code>sigma_xx</code> , <code>sigma_yy</code> , <code>sigma_zz</code> , <code>sigma_xy</code> , <code>sigma_xz</code> , <code>sigma_yz</code>)
Velocity / <code>mpi_rank</code>	In BOTH volume + bulk	In <code>kinematics.vtkhdf</code> only
12 L2-p0 fault projections (<code>slip_dip</code> , <code>slip_strike</code> , <code>slip_rate</code> , <code>traction</code> , <code>state_variable</code> , <code>normal_stress</code> , <code>param_a</code> , <code>param_Dc</code> , <code>fault_x2</code> , <code>fault_x3</code>)	Always in <code>volume.vtkhdf</code>	Default OFF; opt back in via <code>SetRegisterFaultProjectionsInVolumePV(true)</code> (see “Layer 1” below)

Three layers a user can adjust

Output content + cadence + compression are controlled at three layers, from least to most invasive:

Layer 1 — driver code, before the time loop (compile-time) For driver authors. Each TPV driver constructs two `seas::ParaViewOutput<MeshT>` instances:

```
// drivers/tpv102_driver.cpp ~ line 1904
pv_out = std::make_unique<seas::ParaViewOutput<MeshT>>(
    output_dir + "/ParaView", pmesh, order,
    /*collection_name=*/"kinematics", volume_mode);
pv_out->RegisterDomainField("velocity", pv_vel_gf.get());
pv_out->RegisterDomainField("mpi_rank", pv_rank_gf.get());
pv_out->InitFaultOutputBP5(...);           // allocates the 12 GFs but
                                           // does NOT register them
                                           // by default (R-001 fix)

// drivers/tpv102_driver.cpp ~ line 2120 (only when --paraview-bulk-dt > 0)
pv_bulk_out = std::make_unique<seas::ParaViewOutput<MeshT>>(
    output_dir + "/ParaView_bulk", pmesh, order,
    /*collection_name=*/"stress", volume_mode);
pv_bulk_out->RegisterDomainField("sigma_xx", pv_bulk_sxx_gf.get());
pv_bulk_out->RegisterDomainField("sigma_yy", pv_bulk_syy_gf.get());
// ... 4 more sigma components
```

To add a field, add a `GridFunction` and call `RegisterDomainField("name", gf)` on the appropriate collection before any `Save()`. To remove one, comment out its `RegisterDomainField` call. To put the 12 fault projections back into `kinematics.vtkhdf`, call `pv_out->SetRegisterFaultProjectionsInVolumePV(true)` **BEFORE** `InitFaultOutputBP5(...)`. The default is OFF.

The unit test `seas_test_kinematics_field_set` (`tests/unit/test_kinematics_field_set.cpp`) guards both scenarios — flipping the default to true makes the test fail 13/31.

Layer 2 — sbatch CLI flags (per-run) For job submitters. Per-run cadence and ZFP tolerance go on the `ibrun` line in the `sbatch`:

```

ibrun ./seas_tpv102_driver \
    ...
    --paraview                # enable PV output
    --paraview-dt 0.5         # kinematics + fault cadence (s)
    --paraview-bulk-dt 0.5    # stress cadence (s; 0 = no stress file)
    --paraview-volume-zfp-tol 1e-3 # ZFP abs-error on kinematics.vtkhdf
    --paraview-bulk-zfp-tol 1e-3  # ZFP abs-error on stress.vtkhdf
    --paraview-fault-zfp-tol 1e-12 # ZFP abs-error on fault.vtkhdf
    --paraview-max-snapshots 200   # soft cap on total fault writes
    --no-volume-pv              # BP5 production: suppress kinematics

```

The volume flag prefix is still `--paraview-volume-*` and `--paraview-bulk-*` after the refactor — the driver-side rename to `--paraview-kinematics-*` / `--paraview-stress-*` is queued as a follow-up. Same flag semantics; only the collection name on disk changed.

To **drop the stress file entirely**, omit `--paraview-bulk-dt` (the secondary collection isn't constructed). To **drop the kinematics file**, pass `--no-volume-pv`. To skip `fault.vtkhdf`, well... don't pass `--paraview` (everything turns off).

Independent ZFP tolerances are the architectural reason both volume files exist:

Field	Dynamic range	Recommended tol	Why
velocity (kinematics)	~1e-6..10 m/s, mostly O(1)	1e-3	±1 mm/s invisible at coseismic peak
stress components	~1e6..1e8 Pa	1e-3 (Pa)	~10 decades below peak; safe
slip rate (fault)	1e-9..1e0 m/s during nucleation->event	1e-12	3 decades below smallest interesting value

Picking tolerances bigger than these will quantize interesting features. Picking smaller wastes storage with no science benefit.

Layer 3 — TOML config (future, deferred per R-313) For `seas_driver` users, once PV wiring lands there. The planned schema:

```

[paraview]
enabled = true

[paraview.kinematics]
fields   = ["velocity", "mpi_rank"]
zfp_tol  = 1e-3
dt       = 0.5
backend  = "hdf5"

[paraview.stress]
fields   = ["sigma_xx", "sigma_yy", "sigma_zz",
            "sigma_xy", "sigma_xz", "sigma_yz"]
zfp_tol  = 1e-3
dt       = 0.5

[paraview.fault]
zfp_tol  = 1e-12
backend  = "hdf5"

```

Currently `seas_driver.cpp` doesn't construct a `seas::ParaViewOutput` (R-313 deferred deviation; see CLAUDE.md § "Phase 4 deferred deviation"). The TOML schema lands here as a future-use spec; the parsing wiring follows when BP5 production runs through `seas_driver` actually need a `kinematics.vtkhdf`.

Quick recipes

- **"I want stress data but no velocity"** -> add `--paraview-bulk-dt 0.5` to the sbatch, ALSO pass `--no-volume-pv` to suppress `kinematics.vtkhdf`. `stress.vtkhdf` will be the only volume file.
- **"I want lossless ZFP"** -> pass `--paraview-{volume,bulk,fault}-deflate-level 6` (omit the `-zfp-tol` flags). Output is ~2-3× larger.
- **"I want the fault projections in the kinematics file for visualization"** -> in your driver code, call `pv_out->SetRegisterFaultProjectionsInVolumePV(true)` before `InitFaultOutputBP5(...)`. No sbatch change needed. Note this re-introduces ~12× duplicate data with `fault.vtkhdf`.
- **"I want a different sigma component subset"** -> patch the driver: comment out the unwanted `pv_bulk_out->RegisterDomainField("sigma_XX", ...)` calls in `tpv*_driver.cpp` (around line 2141–2148 for `tpv102`). Then the corresponding `memcpy` in `paraview_write` is dead code but harmless.

Architecture

Three back ends, three switches

Build flag	Default fault back end	Default volume back end
MFEM_USE_HDF5=YES	<code>FaultOutputMode::Hdf5</code> -> <code>mfem::ParaViewHDFDataCollection</code> -> <code>fault_surface.vtkhdf</code>	<code>VolumeOutputMode::Hdf5</code> -> <code>mfem::ParaViewHDFDataCollection</code> -> <code>volume.vtkhdf</code>
MFEM_USE_HDF5=NO	<code>FaultOutputMode::Vtu</code> -> single binary VTU/cycle (gather-to-rank-0)	<code>VolumeOutputMode::Vtu</code> -> <code>mfem::ParaViewDataCollection</code> (per-rank VTU)

CLI overrides on every driver (`tpv102`, `tpv104`, `tpv205`): - `--paraview-fault-vtu` / `--paraview-fault-hdf5` — pick fault back end - `--paraview-volume-vtu` / `--paraview-volume-hdf5` — pick volume back end - `--paraview-fault-legacy-ascii` — restore the pre-Phase-1 per-rank ASCII writer (debugging only) - `--no-volume-pv` — suppress the primary volume file entirely (BP5 production default)

Key files

miniapps/seas/io/	
├─ <code>paraview_output.hpp</code>	2093 lines — main wrapper template `seas::ParaViewOutput<MeshType>`
├─ <code>fault_vtu_binary.hpp</code>	735 lines — Phase 1 gather-to-rank-0 binary VTU + <code>LocalFaultPack</code> / <code>GatheredFaultPack</code> data structures
└─ <code>fault_vtkhdf_writer.hpp</code>	304 lines — Phase 2b rank-0 VTKHDF writer for the fault surface
mesh/vtkhdf.{hpp,cpp}	Low-level VTKHDF (HDF5) writer. Used by <code>ParaViewHDFDataCollection</code> . MFEM-level; not seas-specific.

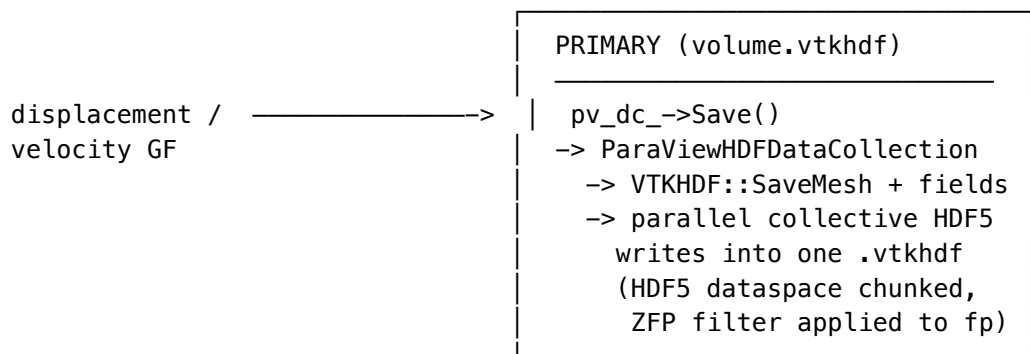
fem/datacollection.hpp/cpp

mfem::ParaViewDataCollection (legacy
per-rank VTU) and
mfem::ParaViewHDFDataCollection
(single-file VTKHDF).

miniapps/seas/drivers/

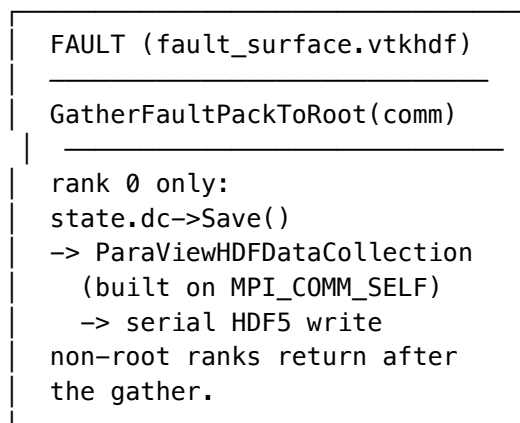
- └─ tpv102_driver.cpp Constructs `pv\_out` and (conditionally)
- └─ tpv104_driver.cpp `pv\_bulk\_out`; calls into the wrapper
- └─ tpv205_driver.cpp inside the time loop.
- └─ seas_bp5_full doesn't construct ParaViewOutput directly; the
BP5 verification driver (tests/verification/bp5_verification_full.cpp)
constructs it and calls Save/WriteFaultSurfaceVTU itself.

Data flow per write cycle (parallel build, HDF5 mode)

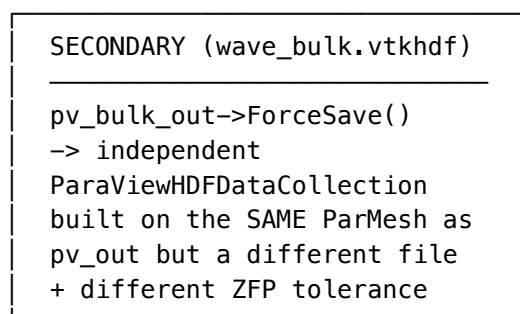


per-DOFData fault
fields (slip, V,
traction, state)

all ranks -> rank 0



σ_{yy} , σ_{xy} , σ_{xz} GFs
(TPV* only)



External dependencies

Library	Used for
HDF5 (parallel)	VTKHDF file substrate. Required: H5_HAVE_PARALLEL => MFEM_PARALLEL_HDF5. ParMesh runs static_assert if missing (paraview_output.hpp:126).
ZFP	Lossy floating-point compression. extern/zfp/install/lib64/libzfp.so.
H5Z-ZFP	HDF5 filter plugin id 32013 that wraps ZFP. Loaded at runtime via HDF5_PLUGIN_PATH; not linked into MFEM.
zlib	Lossless fallback for integer datasets (connectivity, offsets, cell types). Available unconditionally with HDF5.

The extern/ directory is populated by build_frontera.sh; the runtime plugin path is set in each Phase 6 sbatch under miniapps/seas/jobs/{bp5, tpv*}/.

Call Graph (critical path, TPV* driver)

```

tpv*_driver.cpp::main()
  L-> argv parsing (--paraview-* family)
  L-> ParaViewOutput<ParMesh> pv_out(prefix, pmesh, order,
                                     "volume", volume_mode)
  L-> pv_out->RegisterDomainField("velocity", ...)
  L-> pv_out->InitFaultOutputBP5(interior_faces, shared_faces, nbf)
  L-> pv_out->SetFaultHDFCompression(ZfpAccuracy, fault_zfp_tol)
  L-> pv_out->SetVolumeHDFCompression(ZfpAccuracy, volume_zfp_tol)
  L-> pv_out->SetVolumeSaveEnabled(use_volume_pv)
  L-> pv_out->SetVolumePVDt(volume_pv_dt)
  L-> pv_out->GetSchedule().{v_coseismic, dt_coseismic, ...} = CLI overrides
  L-> pv_out->GetSchedule().Validate()
  L-> pv_out->SetTotalRunTime(tfinal) // required for max_total_snapshots
  L-> [optional] pv_bulk_out = ParaViewOutput<ParMesh>(prefix+"/ParaView_bulk",
                                                       pmesh, order, "wave_bulk")

  L-> pv_bulk_out->fixed_dt = bulk_dt
  L-> pv_bulk_out->RegisterDomainField("velocity"/"sigma_yy"/...)
  L-> pv_bulk_out->SetVolumeHDFCompression(ZfpAccuracy, bulk_zfp_tol)
  |
  L-> time loop:
    L-> paraview_write(step_num, time, V_max):
      |-> pv_out->PeekShouldWrite(...) // schedule probe, NO mutation
      |-> pv_bulk_out && pv_bulk_out->PeekShouldWrite(...)
      |-> pack pv_vel_gf, pv_bulk*_gf from solver state
      |-> pv_bulk_out->ForceSave(step_num, time)
      |   L-> pv_dc->Save() (ParaViewHDFDataCollection)
      |       L-> VTKHDF::SaveMesh + AppendField
      |           L-> collective HDF5 writes (chunked + filtered)
      |-> pv_out->UpdateFaultFieldsBP5(...) -> fills L2-p0 fault GFs

```



```

|> pv_out->ForceSave(step_num, time)
|   ↳ ForceSaveImpl: if (emit_volume_save_) pv_dc->Save()
|> pv_out->CommitSchedule(time, V_max) // advance regime FSM
|> pv_out->WriteFaultSurfaceVTU(...):
|   ↳ build LocalFaultPack from per-DOFData
|   ↳ vtu::GatherFaultPackToRoot(local, rank, nranks, comm)
|   ↳ if FaultOutputMode::Hdf5:
|       vtkhdf::WriteFaultPackHdf(state, prefix,
|                                   cycle, time, ...)
|       ↳ rank 0: state.dc->Save() (serial HDF5)
|       ↳ non-root: early return
|   ↳ if FaultOutputMode::Vtu:
|       vtu::WriteFaultPackVTU(path, gathered, BINARY)
|       ↳ rank 0 writes one .vtu, appends PVD entry

```

Call graph (BP5)

seas_bp5_full (= tests/verification/bp5_verification_full.cpp) is similar but:

- No secondary pv_bulk_out.
- Adaptive-schedule (V_max-driven) is the default cadence; --paraview-dt/--paraview-every override it.
- Production runs default to --no-volume-pv (or --paraview-fault-only) — only fault_surface.vtkhdf and station-CSV are emitted.

Detailed Walkthrough

seas::ParaViewOutput<MeshType> (paraview_output.hpp:69 — 2093 lines)

Template parameter MeshType $\in \{\text{Mesh}, \text{ParMesh}\}$ selects the corresponding GridFunction / FiniteElementSpace types via the GFTrait trait at the top of the file.

Construction

```

ParaViewOutput(prefix, mesh, order,
               collection_name = "volume",
               mode = DefaultVolumeOutputMode())

```

- prefix — output directory.
- mesh — domain mesh (held by reference; not owned).
- order — polynomial order, passed to SetLevelsOfDetail.
- collection_name — base filename for HDF5 mode (<prefix>/<name>.vtkhdf) or subdir for VTU mode.
- mode — locks the volume back end for the lifetime of the wrapper.

DefaultVolumeOutputMode() is a static constexpr function that returns Hdf5 whenever the build supports it for the current MeshType, otherwise Vtu. For ParMesh on a build without MFEM_PARALLEL_HDF5, it triggers a static_assert rather than silently falling back to VTU (R-401, paraview_output.hpp:126–131). This is intentional: the silent fallback used to mask every Frontera ParMesh run dropping into the legacy per-rank VTU path.

Three writers, three responsibilities

Member	Type	When written	Lifecycle
pv_dc_	unique_ptr<ParaViewDataCollector> (pointing at either ParaViewDataCollection or ParaViewHDFDataCollection)	when emit_volume_save_ AND the user dt elapsed	constructed in the ctor; survives the run
fault_hdf_state_	vtkhdf::FaultHDFState (rank-0 only)	every WriteFaultSurfaceVTU when FaultOutputMode::Hdf5	constructed lazily on first write
(binary VTU path)	none — stateless emit via vtu::WriteFaultPackVTU	every WriteFaultSurfaceVTU when FaultOutputMode::Vtu	n/a

The fault-surface and primary-volume writers DO NOT share files. They share the schedule (adaptive_, last_write_time_, current_regime_) but their on-disk artefacts are independent.

Schedule state machine — AdaptiveSchedule A 3-regime finite state machine keyed on the global max slip rate $V_{\max}$:

regime 0 (interseismic)	$V \leq v_{\text{nucleation}}$	interval = dt_interseismic (default 1 yr)
regime 1 (nucleation)	$v_{\text{nucleation}} < V \leq v_{\text{coseismic}}$	interval = dt_nucleation (default 1 s)
regime 2 (coseismic)	$V > v_{\text{coseismic}}$	interval = dt_coseismic (default 0.01 s)

State transitions in NextRegime(V , prev_regime): - Entries use strict $>$ comparisons. - Exits use $V / \text{hysteresis_factor} (\geq 1.0)$. hysteresis_factor = 1.0 (default) disables hysteresis. - Non-finite $V \rightarrow$ regime 2 (so we keep dense output during a blowup). - “Drop-past-threshold”: a coseismic- $>V$ drop that lands in the nucleation band skips regime 1 and goes directly to interseismic. Rationale at paraview_output.hpp:232–243: avoids over-sampling post-rupture ring-down.

Three save methods, three semantics

Method	Mutates schedule	Calls pv_dc_→Save ()	When to use
Save(cycle, time, $V_{\max}$)	yes (regime + last_write_time + last_v_max)	yes (subject to emit_volume_save_)	Standard one-shot “schedule decides + write everything”
ShouldWrite(cycle, time, $V_{\max}$)	yes	no	“Schedule decides but skip the volume” — fault-only with the snapshot counter still bumping
PeekShouldWrite(cycle, time, $V_{\max}$) const	no	no	Schedule probe before doing expensive packing work. Must be followed by CommitSchedule(time, $V_{\max}$) to advance state

Method	Mutates schedule	Calls <code>pv_dc_</code> → <code>Save()</code>	When to use
<code>ForceSave(cycle, time)</code>	bumps <code>last_write_time_</code> + snapshot counter	yes (subject to <code>emit_volume_save_</code>)	Caller decides (“write now”) — bypass schedule
<code>CommitSchedule(time, V_max)</code>	yes	no	Advance schedule state without writing volume (used after <code>PeekShouldWrite + WriteFaultSurfaceVTU</code> only)

The TPV* drivers use `PeekShouldWrite` → `ForceSave`/`CommitSchedule` → `WriteFaultSurfaceVTU` so they can decide whether to copy fault data into the registered GFs (an $O(\text{num_dofs})$ memcopy) on each step.

Snapshot cap — `max_total_snapshots` If set (default 0 = uncapped), `SnapshotCapAwareInterval` inflates `dt_interseismic` to keep the projected total snapshot count under the cap:

```
remaining_budget = K - snapshots_so_far
new_dt_inter      = max(dt_inter, (T - t) / remaining_budget)
```

Coseismic and nucleation cadences are never inflated; they’re physically meaningful. The cap is “soft” — a long-running event may push the final count slightly over K (≤ 1 extra per event, per design).

The cap requires `SetTotalRunTime(tfinal)` — without it the cap is a no-op and the constructor asserts loudly rather than silently degrading.

Fault-side state — `InitFaultOutputBP5` and `UpdateFaultFieldsBP5` `InitFaultOutputBP5(fault_interior_faces, fault_shared_faces, nbf_per_face)` constructs the L2-p0 fault GridFunctions on the **domain** mesh:

- 8 scalar GFs: `fault_slip_{dip,strike}`, `fault_slip_rate_{dip,strike}`, `fault_trac_{dip,strike}`, `fault_state`, `fault_normal_stress`.
- One DOF per element (L2 $p=0$).
- Fault values are scattered from the per-face DOFData layout to the two volume elements adjacent to each fault face — only elements touching a fault face carry nonzero values.
- For shared faces, only the local-side `Elem1` carries the value (`Elem2` lives on a neighbour rank).

These GFs are registered with the primary `pv_dc_` so they appear in `volume.vtkhdf` alongside displacement/velocity. They are NOT the same data the fault-surface writer emits — see “Gotchas” below.

`UpdateFaultFieldsBP5` is called every output cycle to refresh the GF values from the latest DOFData snapshot.

Fault-surface writer — `WriteFaultSurfaceVTU` Five branches:

1. **Legacy ASCII** (`legacy_ascii_ = true`): one ASCII VTU per rank per cycle + per-cycle PVTU + cumulative PVD. Pre-Phase-1 layout. Kept verbatim because four unit tests depend on the ASCII regex shape (`test_fault_surface_vtu_continuity` and friends).
2. **Phase 1 binary VTU** (`output_mode_ = Vtu`, default on `MFEM_USE_HDF5=N0`): every rank packs a `vtu::LocalFaultPack`, MPI-gathers to rank 0 via `GatherFaultPackToRoot`, rank 0 writes ONE `<prefix>/FaultSurface/fault_surface_c<cycle>.vtu` (binary, base64-inline), appends a PVD entry, writes PVD index. File count drops from $\text{n ranks} \times \text{ncycles}$ to $1 \times \text{ncycles}$.

3. **Phase 2b VTKHDF** (`output_mode_ = Hdf5`, default on HDF5 builds): same gather, but rank 0 calls `vtkhdf::WriteFaultPackHdf` which feeds the gathered pack into a `ParaViewHDFDataCollection` constructed against a serial in-memory `mfem::Mesh` (one triangle per fault face). One `.vtkhdf` file for the entire run.
4. **`has_fault_output_ == false`** (collective early return): all ranks return — the wrapper was constructed but `InitFaultOutputBP5` was never called.
5. **Compile-time guards:** on `MFEM_USE_HDF5=N0` builds, `output_mode_ == Hdf5` aborts at runtime; CLI arg parsing in the drivers rejects `--paraview-fault-hdf5` at parse time so the abort is unreachable from external callers.

Uniformity check — `paraview_output.hpp:1076–1095` The first `WriteFaultSurfaceVTU` call on a parallel build broadcasts (`legacy_ascii_`, `output_mode_`) from rank 0 and asserts every rank matches. If `SetFaultOutputMode/SetLegacyAsciiVTU` are called from a single rank only, the gather collective would deadlock; this check converts the deadlock into a loud abort. The check is cached (`output_mode_uniformity_checked_`) so the broadcast doesn't fire 4M times on a 5000-cycle BP5 run.

`vtkhdf::FaultHDFState` and `WriteFaultPackHdf` (`fault_vtkhdf_writer.hpp`)

The Phase 2b fault writer **deliberately deviates from the plan**. The plan specified a `ParSubMesh`-based per-rank fault submesh + collective parallel HDF5 write. The deviation note at `fault_vtkhdf_writer.hpp:22–47` explains:

- `ParSubMesh::CreateFromBoundary` requires the submesh source to be a boundary attribute; fault faces are *interior*. Promoting them to boundary would invalidate the elasticity operator's attribute contract.
- The fault surface is at most a few thousand triangles even at BP5 scale; parallel-HDF5 throughput gain is negligible vs reusing the already-tested Phase 1 gather.
- The on-disk result is identical: one `.vtkhdf` file per run.

Implementation:

- All ranks call `GatherFaultPackToRoot(local, rank, nrank, comm)` collectively. Non-root ranks then return.
- Rank 0 constructs a serial 2D-in-3D `mfem::Mesh` of triangles on first call (`Init(prefix, g)`), with one triangle per fault face, L2-p0 finite element space, one `GridFunction` per emitted scalar field, and a `ParaViewHDFDataCollection` on the serial mesh.
- Subsequent calls reuse the mesh + GFs (geometry is static across cycles, asserted via `MFEM_VERIFY`).
- The chunk filter (Deflate level OR ZFP tolerance) is set on the data collection at Init time; cannot change mid-run.
- Each cycle: copy gathered field values into the L2-p0 GFs, `dc->SetCycle(cycle); dc->SetTime(time); dc->Save();`. Inside MFEM, `ParaViewHDFDataCollection::EnsureVTKHDF` dynamic_casts the mesh as serial, so HDF5 calls are non-collective (`fem/datacollection.cpp:1374–1402`).

`vtu::LocalFaultPack / GatherFaultPackToRoot` (`fault_vtu_binary.hpp`)

Three data structures + three helpers:

Type	Layout
LocalFaultPack	vertices[3*ncells], triangles[ncells]={3i,3i+1,3i+2}, field_arrays[k][ncells], field_names[k]
GatheredFaultPack	Concatenations of per-rank LocalFaultPacks with triangle indices remapped to point at the global vertex array

Invariant: `triangles[i] == {3i, 3i+1, 3i+2}` — each triangle owns three unique vertices, no inter-cell sharing. This matches the legacy per-face emit and makes the gather's index remap a constant offset add.

The gather is collective on the supplied communicator (or a no-op on serial builds). The field-name list is OR-reduced across ranks so every rank packs the same fields — the `_k4` diagnostic fields are present iff at least one rank has them.

`WriteFaultPackVTU(path, g, format, compression_level)` writes the gathered pack as a binary-with-base64-inline VTU. The format deviation (inline-base64 vs raw-appended) is documented at `fault_vtu_binary.hpp:26–41` — chose inline-base64 because (a) it reuses MFEM's already-tested writers and (b) `MFEM_USE_ZLIB=YES` recovers the size delta vs raw-appended.

MFEM-level ParaViewHDFDataCollection (fem/datacollection.{hpp,cpp})

The MFEM-side counterpart of `ParaViewDataCollection`. The class:

```
class ParaViewHDFDataCollection : public ParaViewDataCollectionBase {
    std::unique_ptr<VTKHDF> vtkhdf;
    void EnsureVTKHDF();
    template <typename FP_T> void TSave();
public:
    ParaViewHDFDataCollection(name, mesh = nullptr);
    void SetCompression(bool) override;
    enum class HDFCompression { Deflate, ZfpAccuracy };
    void SetHDFCompression(HDFCompression alg, double param);
    void Save() override;
};
```

- One `.vtkhdf` file per collection, lazily constructed on first `Save()` via `EnsureVTKHDF()`.
- Two compression modes:
 - Deflate (default at level 6): lossless zlib (preceded by HDF5 byte shuffle).
 - ZfpAccuracy: LLNL ZFP filter id 32013 at absolute-error tolerance param. Applied **only** to FP32/FP64 datasets; integer connectivity falls back to lossless deflate via `VTKHDF::EnsureDataset`.
- Filter selection is captured on the collection and applied to the underlying VTKHDF on the next `Save()`. It is **locked at first save** — calling `SetHDFCompression` after the first save updates the collection state but does not retroactively re-filter previously-written timesteps.

MFEM-level VTKHDF (mesh/vtkhdf.{hpp,cpp})

Low-level VTKHDF writer. The seas miniapp doesn't call it directly; everything goes through `ParaViewHDFDataCollection`. Useful facts for debugging:

- One HDF5 file per VTKHDF instance. Opens with `H5F_ACC_TRUNC` by default (no restart yet; planned follow-up).

- Hierarchy: /VTKHDF/{Points, Connectivity, Offsets, Types, NumberOfPoints, NumberOfCells, NumberOfConnectivityIds, PointData/<name>, CellData/<name>, Steps, Steps/PartOffsets, ...}.
- Datasets grow on the first dimension via H5S_UNLIMITED. Each Save() extends and appends.
- Chunked storage with chunk size targeting ~0.5 MB. Filter chain attached at EnsureDataset time (lossless for integers, ZFP-or-deflate for floats).
- Phase 2d requirements pulled into MFEM proper (H5S_BLOCK, unsigned long long hsize_t) bumped the minimum HDF5 to **1.14.x**. On Frontera, this forced a from-source build at extern/hdf5/install (the intel-19-compatible module caps at 1.12.2).
- A type-id template TypeID<T> maps C++ types to HDF5 type ids. hsize_t's underlying type changed from uint64_t (= unsigned long on Linux x86_64) to unsigned long long in 1.14, requiring exactly that specialization (mesh/vtkhdf.cpp:32–36).

build_frontera.sh + Phase 6 sbatch glue

The build- and run-time substrate that makes the Phase 6 path work on Frontera. None of this is in the C++ source — but the C++ won't link or run without it.

build_frontera.sh (project root) builds three from-source extern installs:

1. **HDF5 1.14.6** -> extern/hdf5/install (parallel build against intel/19 + impi/19; Frontera's phdf5 module caps at 1.12.2 which is too old).
2. **ZFP 1.0.1** -> extern/zfp/install (LLNL's reference encoder/decoder; built with CMake, mpicc, intel/19).
3. **H5Z-ZFP v1.1.1** -> extern/h5z-zfp/install/plugin/libh5zzfp.so (HDF5 filter plugin id 32013 that wraps ZFP; built with make HDF5_HOME=... ZFP_HOME=...).
4. **gmsh 4.13.1** -> extern/gmsh/bin/gmsh (built from source with gcc/g++; needed only for .geo -> .msh regen at sbatch submit time, not for MFEM/seas itself).

MFEM is then configured with:

```
MFEM_USE_HDF5=YES
MFEM_USE_H5Z_ZFP=YES          # required so #define lands in _config.hpp
HDF5_OPT=-I${HDF5_INSTALL}/include
HDF5_LIB=-L${HDF5_INSTALL}/lib -lhdf5_hl -lhdf5 -lz
```

The H5Z-ZFP plugin is **not** linked into MFEM. HDF5 discovers it at runtime via HDF5_PLUGIN_PATH. Each Phase 6 sbatch sets:

```
export HDF5_PLUGIN_PATH="${SEAS_MFEM_ROOT}/extern/h5z-zfp/install/plugin"
```

If the plugin isn't on HDF5_PLUGIN_PATH at run time, HDF5 fails the first ZFP write with "required filter is not registered". seas::ParaViewOutput::ProbeH5ZZfpPluginOrAbort() (paraview_output.hpp:503–546) probes the env-var before the first save and aborts with an actionable message instead.

Phase chronology (from git log, document/io_dev/PLAN_*.md)

Phase	Commit	What landed
Pre-Phase-1	297d249	Initial per-rank ASCII VTU + cumulative PVD for BP5.
	1cf3291–c409afb	Misc tightening (fault PVD time series, friction parameter fields, per-DOF mapping audit).

Phase	Commit	What landed
Phase 1	079e348	--paraview-dt X (fixed time interval, overrides step interval).
	a78c623, b2b9531	Adaptive V_max schedule with hysteresis (BP5 only).
	4f856e7	Gather-to-rank-0 binary appended VTU writer. One file per cycle instead of nranks files per cycle.
Phase 2b	(same WIP)	Single-file VTKHDF for the fault. Rank-0-only serial HDF5 inside a ParaViewHDFDataCollection on MPI_COMM_SELF.
Phase 2d	de73e7d (folded in)	H5Z-ZFP lossy compression on the fault path. Adds
Phase 3	(folded in)	--paraview-fault-zfp-tol. max_total_snapshots cap + --paraview-
Phase 4	(folded in)	{co,nucleation,inter}seismic-dt CLI overrides.
		--no-volume-pv / --paraview-fault-only, decoupling fault from volume PV.
Phase 6	de73e7d	Primary volume goes VTKHDF by default on MFEM_USE_HDF5=YES builds. --paraview-volume-hdf5/--paraview-volume-vtu CLI selector. SetVolumeHDFCompression(...) actually applies. Secondary pv_bulk_out reroutes --paraview-bulk-* from pv_out to a new collection in <pre-fix>/ParaView_bulk/wave_bulk.vtkhdf.
Phase 6 runtime fixes	(current session, multiple commits)	HDF5 1.14.6 from-source on Frontera; gmsh from-source; module load python3; SEAS_MFEM_ROOT auto-detect in sbatch; MFEM_USE_H5Z_ZFP placeholder in config/config.mk.in.

The bulk-compression plan (PLAN_bulk_compression_and_size_estimator_2026-05-09.md) is the Phase 6 spec; the older PLAN_paraview_compaction_2026-04-28.md carries Phases 0-5.

Conventions

- **MeshType templating.** Every method that touches mesh data is a template on MeshType (Mesh / ParMesh) via the GFTrait trait. Serial path is the parallel path with all MPI calls compiled away.

- **Collective vs rank-0.** `Save()`, `ShouldWrite()`, `CommitSchedule()`, `WriteFaultSurfaceVTU()` are all collective: every rank must call. The fault file content is rank-0-only but reaching the rank-0 write requires a collective gather.
 - **No owning pointers.** Mesh, fault face arrays, DOFData vectors are all passed by reference and not owned. The wrapper owns only its own GFs (`fault_*_`), the underlying data collections (`pv_dc_`), and the H5Z-ZFP probe state.
 - **CLI parsing.** Drivers use ad-hoc `HasFlag/GetIntArg/GetRealArg/GetStringArg` helpers (defined in each `tpv*_driver.cpp`). No `argparse`-style library; the flag names are kept identical across all four drivers by manual mirroring.
 - **Error handling.** `MFEM_VERIFY` (always-on) for invariants; `MFEM_ABORT` for unreachable branches. `mfem::out` for rank-0 prints (no-op on non-root); `std::cout` for per-rank.
 - **Filter locking.** `SetFaultHDFCompression` / `SetVolumeHDFCompression` MUST be called before the first `Save()` on the corresponding collection — the filter selection is read by `VTXHDF::EnsureDataset` exactly once when the first dataset is created. Later calls are accepted but only take effect on **new** datasets (which there generally aren't, since the file is built up entirely on the first save).
-

Gotchas

- **Two fault representations coexist.** `WriteFaultSurfaceVTU` emits **one triangle per fault face** in `fault_surface.vtkhdf`. `UpdateFaultFieldsBP5` + the primary volume save emit **L2-p0 cell data on the volume mesh** (one DOF per *volume* element, scattered to elements adjacent to fault faces). The fault file is the science output; the volume's `fault_*` fields are debug rendering. They will differ near shared-face boundaries because L2-p0 averages across faces.
- **(RESOLVED 2026-05-12)** Earlier versions of the codebase wrote `velocity` and `mpi_rank` into BOTH `volume.vtkhdf` and `wave_bulk.vtkhdf` — byte-for-byte duplication when cadences/tolerances matched (which they did in the Phase 6 sbatch defaults). The `split-bulk-solutions` refactor (see [document/io_dev/PLAN_split_bulk_solutions_2026-05-12.md](#)) fixed this:
 - `kinematics.vtkhdf` (renamed from `volume.vtkhdf`) holds `velocity` + `mpi_rank` (no stresses, no fault projections).
 - `stress.vtkhdf` (renamed from `wave_bulk.vtkhdf`) holds the full symmetric stress tensor (6 components: `sigma_xx`, `_yy`, `_zz`, `_xy`, `_xz`, `_yz`) and **no** `velocity` / `mpi_rank`.

Consequence for downstream users: if you open just `stress.vtkhdf` in ParaView to view a stress wavefield, there's no longer a `velocity` field in the same view — you'd need to also open `kinematics.vtkhdf` and use *Group Datasets* if you want both. This is by design: the duplication cost wasn't worth the convenience.
- **L2-p0 fault GFs need `nb_f_per_face > 0`.** `InitFaultOutputBP5` writes per-face averages `inv_nbf = 1.0 / nb_f_per_face_`; `nb_f=0` would silently produce NaN `CellData`. R-007 added a hard `MFEM_VERIFY` at the call boundary.
- **`fix_orientation=true` in the rank-0 fault mesh** (`fault_vtkhdf_writer.hpp:169`) — `gmsh` may reorder triangle vertices to enforce a positive Jacobian. This is safe for **scalar** cell data (the only thing this writer currently emits). If anyone adds vector cell data or per-vertex data, the gathered (`3i`, `3i+1`, `3i+2`) ordering will no longer match the rendered vertex order (R-111 note inline).
- **`PeekShouldWrite` then `CommitSchedule(time)` (single-arg) is only correct with `hysteresis_factor = 1.0`.** If hysteresis is enabled the regime FSM is not memoryless in `V`, so the single-arg shim uses the **last** `V_max` recorded by `Save/ShouldWrite` rather than the *current* `V_max`. Hysteresis-enabled callers MUST use `CommitSchedule(time, V_max)`.
- **`current_regime_advancing` only when something writes.** If a fault-only path skips `pv_→Save()`

and forgets `CommitSchedule`, `current_regime_` pins at 0 and hysteresis silently no-ops. The two-arg `CommitSchedule(time, V_max)` is the cure.

- **max_total_snapshots requires SetTotalRunTime(tfinal).** Without it, the cap is silently a no-op. `SnapshotCapAwareInterval` aborts with a readable message rather than degrading.
 - **--paraview-bulk-* flag scope changed in Phase 6.4.** Pre-Phase-6 those flags applied to the primary `pv_out` (with a one-time rank-0 warning that they didn't work on the VTU path). Post-Phase-6 they route to the **secondary** `pv_bulk_out` collection. BP5 has no secondary, so `--paraview-bulk-*` on BP5 is now a one-shot warning that the flag has no effect.
 - **Default volume back end is HDF5, even when you didn't ask.** On any build with `MFEM_USE_HDF5=YES` + (`MFEM_PARALLEL_HDF5=YES` for ParMesh), the volume collection writes `volume.vtkhdf`, not a per-rank VTU. Downstream scripts that `grep <prefix>/ParaView/<basename>_<rank>_<cycle>.vtu` paths break silently. Pass `--paraview-volume-vtu` to recover the legacy layout.
 - **MFEM_USE_H5Z_ZFP=YES was missing from config/config.mk.in** (now fixed). MFEM's build did `#define MFEM_USE_H5Z_ZFP` in `_config.hpp` but `config.mk` was missing the corresponding `MFEM_USE_H5Z_ZFP = YES` line. The Phase 6 sbatch had a `grep -q '^MFEM_USE_H5Z_ZFP *=YES' config.mk || exit` gate that therefore failed on every run. Fixed in `config.mk.in` to add the placeholder.
 - **HDF5_PLUGIN_PATH is required at run time.** The H5Z-ZFP plugin is not linked into MFEM; HDF5 loads it via this env var. If it's unset or wrong, the first ZFP write fails inside HDF5's filter pipeline (cryptic "required filter is not registered"). The wrapper probes early and aborts with an actionable message.
 - **Filter selection is locked at first save.** Calling `SetVolumeHDFCompression(ZfpAccuracy, 1e-3)` after `pv_out->Save()` has fired once is silently a no-op for the existing datasets. The wrapper makes the call eagerly in the constructor path; if you ever add a "change compression mid-run" flow, the only way to honour it is to close + reopen the file with a new MFEM `ParaViewHDFDataCollection`.
 - **Mid-run mesh swap is unsupported on the Hdf5 path.** The `.vtkhdf` file is keyed on the first mesh handed to the writer; rewiring `pv_dc->SetMesh(new_mesh)` would invalidate the dataset shapes. Document at `paraview_output.hpp:384-386`.
 - **Reruns overwrite .vtkhdf files.** No restart-mode wiring yet; the file is opened with `H5F_ACC_TRUNC` on every run. Planned follow-up.
-

Open Questions

- **pv_bulk_out calls Save()-like methods directly via ForceSave,** bypassing `WriteFaultSurfaceVTU`. There is no equivalent `SetVolumeSaveEnabled(false)` path for the secondary collection — you'd have to not construct it. Is the asymmetry intentional? (My guess: yes, because the bulk is "volume only" by definition. Worth a comment in the driver.)
- **Phase 5 (post-run packaging) is not yet shipped** per the plan doc. There's no tool that takes an already-emitted multi-million-file directory and re-emits it as VTKHDF. Users with pre-Phase-2 output have to re-run.
- **SetTotalRunTime(tfinal) requirement isn't enforced at construction time.** It only matters once `max_total_snapshots > 0`, and only at the moment `SnapshotCapAwareInterval` first runs. If you set `max_total_snapshots` AND forget `SetTotalRunTime`, the abort fires inside the time loop, not at setup. A constructor-time invariant check would be friendlier.
- **The bulk collection inherits the same VolumeOutputMode as the primary.** The TPV drivers pin `pv_bulk_out`'s mode to whatever `pv_out` chose. If you want a per-collection back-end choice

(e.g. lossless deflate on bulk velocity, lossy ZFP on bulk stresses), the API doesn't currently allow that — you'd need to split the bulk into two more collections.

- **output_mode_uniformity_checked_only covers legacy_ascii_ + output_mode_.** Other state that must be uniform across ranks (filter algorithm, ZFP tolerance, snapshot cap) is not checked. In practice these are set from CLI parsing which is uniform, but the assumption isn't enforced.
- **MFEM_USE_H5Z_ZFP doesn't actually gate any link-time code** — it's pure preprocessor, used by `seas::ParaViewOutput::ProbeH5ZZfpPluginOrAbort` to know whether to probe. The flag exists mostly so the sbatch grep gate can fail fast. Could be inlined.